



**FACULTAD DE INGENIERÍA
DEPARTAMENTO DE INGENIERÍA DE SISTEMAS**

Introducción a los Sistemas Distribuidos
Profesor: John Corredor, Ph.D.

**TALLER 04
HILOS Y SOCKETS**

Informe de Laboratorio

Presentado por:

Juan Diego Ariza
Juan Diego Pardo
Juan Sebastian Urbano

Bogotá D.C., Colombia
Mayo de 2026

Índice

1. Introducción	2
2. Objetivos	2
2.1. Objetivo general	2
2.2. Objetivos específicos	2
3. Entorno de Pruebas	2
4. Parte I — Sockets TCP y UDP	4
4.1. Marco Teórico	4
4.2. Implementación	4
4.3. Compilación	5
4.4. Funcionamiento TCP	5
4.5. Funcionamiento UDP	5
4.6. Prueba clave: comportamiento sin servidor	6
4.7. Comparativa TCP vs UDP	7
4.8. ¿Cuándo usar cada uno?	7
5. Parte II — Ejecución Secuencial vs Paralela con Hilos	8
5.1. Caso de Estudio	8
5.2. Las Tres Versiones Implementadas	8
5.2.1. Versión Secuencial (Main.java)	8
5.2.2. Versión con extends Thread (MainThread.java y CajeraThread.java)	8
5.2.3. Versión con implements Runnable (MainRunnable.java)	8
5.3. Resultados Experimentales	9
5.3.1. Ejecución Secuencial	9
5.3.2. Ejecución con Hilos (extends Thread)	10
5.3.3. Ejecución con Runnable	11
5.4. Tabla Resumen de Tiempos	11
5.5. Análisis de los Resultados	13
6. Conclusiones	13
7. Observaciones	14

1. Introducción

El presente informe documenta el desarrollo del Taller 04 de la asignatura Introducción a los Sistemas Distribuidos. El taller plantea dos ejercicios complementarios: por un lado, comparar los sockets **TCP** y **UDP** implementados en Java, observando cómo funcionan, en qué se diferencian y cuándo conviene usar cada uno; y por otro lado, analizar la diferencia entre una ejecución **secuencial** y una ejecución **paralela** usando **hilos** en Java, mediante un caso práctico de cajas procesando clientes en un supermercado.

Ambos temas son pilares de los sistemas distribuidos: los sockets son el mecanismo básico de comunicación entre procesos en red, y los hilos son la herramienta que permite a un mismo proceso atender múltiples tareas en paralelo, aprovechando los procesadores multinúcleo modernos. Comprender sus diferencias y limitaciones es indispensable antes de abordar arquitecturas distribuidas más complejas.

2. Objetivos

2.1. Objetivo general

Comprender en la práctica las diferencias de comportamiento entre los protocolos TCP y UDP, y entre la ejecución secuencial y paralela con hilos en Java.

2.2. Objetivos específicos

- Implementar y ejecutar un cliente y servidor TCP, así como un cliente y servidor UDP, en Java, observando su comportamiento real.
- Identificar las diferencias funcionales entre TCP y UDP en cuanto a conexión, fiabilidad, orden de mensajes y detección de fallos.
- Implementar el caso práctico del supermercado en sus tres versiones: secuencial, con hilos por herencia (*extends Thread*) y con hilos por interfaz (*implements Runnable*).
- Medir y comparar los tiempos de ejecución de las tres versiones.
- Analizar los resultados y formular recomendaciones sobre cuándo usar cada enfoque.

3. Entorno de Pruebas

Las pruebas se realizaron sobre la máquina virtual del laboratorio del Departamento de Ingeniería de Sistemas, cuya configuración se resume en la Tabla 1.

Componente	Descripción
Sistema operativo	Ubuntu Linux 24.04 LTS (VM laboratorio MIG29)
Lenguaje	Java
Compilador / Runtime	OpenJDK 21.0.10
Herramienta de build	GNU Make
Medición de tiempos	<code>/usr/bin/time</code> y <code>System.currentTimeMillis()</code>
Editor	Visual Studio Code

Cuadro 1: Entorno donde se ejecutaron las pruebas.

4. Parte I — Sockets TCP y UDP

4.1. Marco Teórico

Un **socket** es un punto final de comunicación entre dos procesos a través de la red, identificado por una dirección IP y un puerto. En la pila TCP/IP existen dos protocolos principales de la capa de transporte:

TCP (Transmission Control Protocol): protocolo orientado a conexión. Antes de enviar datos se establece una conexión mediante el *three-way handshake* (SYN, SYN-ACK, ACK). Garantiza que los datos lleguen completos, en orden y sin duplicados, retransmitiendo automáticamente lo que se pierde. A cambio, introduce más latencia y consume más recursos.

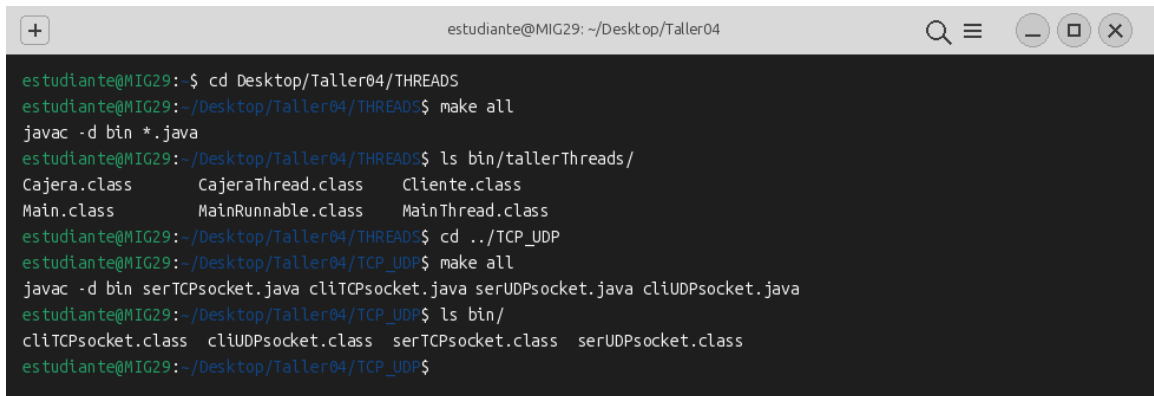
UDP (User Datagram Protocol): protocolo no orientado a conexión. Cada mensaje (datagrama) se envía de forma independiente, sin establecer conexión previa y sin garantizar entrega ni orden. Es más rápido y ligero, pero la aplicación debe asumir las posibles pérdidas o reordenamientos.

4.2. Implementación

Se implementaron cuatro programas en Java, ubicados en la carpeta TCP_UDP/:

- **serTCPsocket.java:** servidor TCP que escucha en el puerto 6001. Crea un `ServerSocket`, espera la conexión del cliente con `accept()` y lee los mensajes con `readUTF()`.
- **cliTCPsocket.java:** cliente TCP que se conecta al servidor mediante `new Socket(addr, 6001)` y envía los mensajes leídos por consola con `writeUTF()` hasta que el usuario escribe `fin`.
- **serUDPsocket.java:** servidor UDP que abre un `DatagramSocket` en el puerto 6000 y recibe datagramas con `receive()`.
- **cliUDPsocket.java:** cliente UDP que envía datagramas al servidor mediante `send()` sin establecer conexión previa.

4.3. Compilación

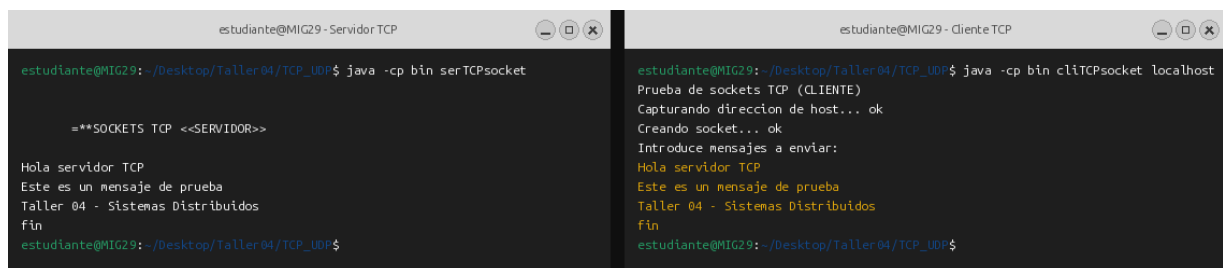


```
estudiante@MIG29: ~/Desktop/Taller04
estudiante@MIG29:~$ cd Desktop/Taller04/THREADS
estudiante@MIG29:~/Desktop/Taller04/THREADS$ make all
javac -d bin *.java
estudiante@MIG29:~/Desktop/Taller04/THREADS$ ls bin/tallerThreads/
Cajera.class      CajeraThread.class  Cliente.class
Main.class        MainRunnable.class  MainThread.class
estudiante@MIG29:~/Desktop/Taller04/THREADS$ cd ../TCP_UDP
estudiante@MIG29:~/Desktop/Taller04/TCP_UDP$ make all
javac -d bin serTCPsocket.java cliTCPsocket.java serUDPsocket.java cliUDPsocket.java
estudiante@MIG29:~/Desktop/Taller04/TCP_UDP$ ls bin/
cliTCPsocket.class cliUDPsocket.class serTCPsocket.class serUDPsocket.class
estudiante@MIG29:~/Desktop/Taller04/TCP_UDP$
```

Figura 1: Compilación de los archivos del taller con Make.

4.4. Funcionamiento TCP

Tras compilar, se ejecuta el servidor en una terminal y el cliente en otra. El servidor recibe los mensajes en el mismo orden en que el cliente los envía, lo que confirma la entrega ordenada característica de TCP.



```
estudiante@MIG29 - Servidor TCP
estudiante@MIG29:~/Desktop/Taller04/TCP_UDP$ java -cp bin serTCPsocket

==**SOCKETS TCP <<SERVIDOR>>

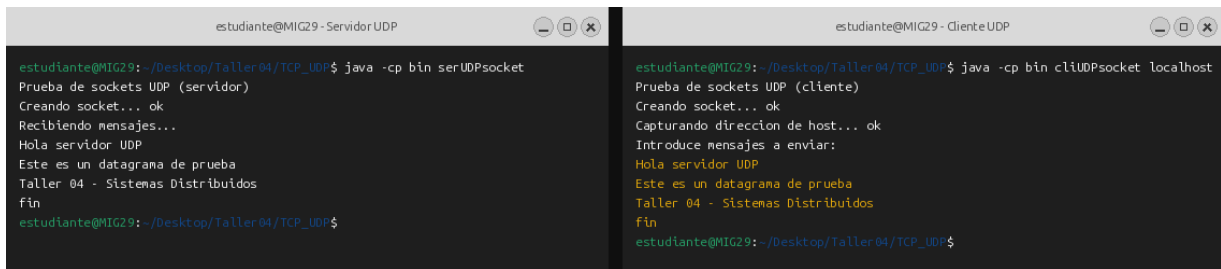
Hola servidor TCP
Este es un mensaje de prueba
Taller 04 - Sistemas Distribuidos
fin
estudiante@MIG29:~/Desktop/Taller04/TCP_UDP$

estudiante@MIG29 - Cliente TCP
estudiante@MIG29:~/Desktop/Taller04/TCP_UDP$ java -cp bin cliTCPsocket localhost
Prueba de sockets TCP (CLIENTE)
Capturando direccion de host... ok
Creando socket... ok
Introduce mensajes a enviar:
Hola servidor TCP
Este es un mensaje de prueba
Taller 04 - Sistemas Distribuidos
fin
estudiante@MIG29:~/Desktop/Taller04/TCP_UDP$
```

Figura 2: Cliente y servidor TCP intercambiando mensajes.

4.5. Funcionamiento UDP

El cliente UDP no establece conexión previa: simplemente empaqueta el mensaje en un `DatagramPacket` con la dirección del destino y lo envía. El servidor recibe los datagramas y los imprime.



```

estudiante@MIG29 - Servidor UDP
estudiante@MIG29: ~/Desktop/Taller04/TCP_UDP$ java -cp bin serUDPsocket
Prueba de sockets UDP (servidor)
Creando socket... ok
Recibiendo mensajes...
Hola servidor UDP
Este es un datagrama de prueba
Taller 04 - Sistemas Distribuidos
fin
estudiante@MIG29: ~/Desktop/Taller04/TCP_UDP$

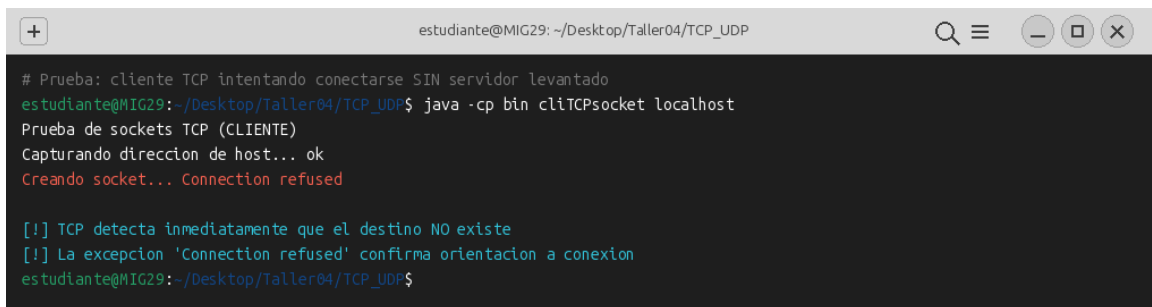
estudiante@MIG29 - Cliente UDP
estudiante@MIG29: ~/Desktop/Taller04/TCP_UDP$ java -cp bin cliUDPsocket localhost
Prueba de sockets UDP (cliente)
Creando socket... ok
Capturando direccion de host... ok
Introduce mensajes a enviar:
Hola servidor UDP
Este es un datagrama de prueba
Taller 04 - Sistemas Distribuidos
fin
estudiante@MIG29: ~/Desktop/Taller04/TCP_UDP$

```

Figura 3: Cliente y servidor UDP intercambiando datagramas.

4.6. Prueba clave: comportamiento sin servidor

Para evidenciar la diferencia más importante entre los dos protocolos, se ejecutaron los clientes **sin levantar previamente el servidor**. El resultado es contundente:

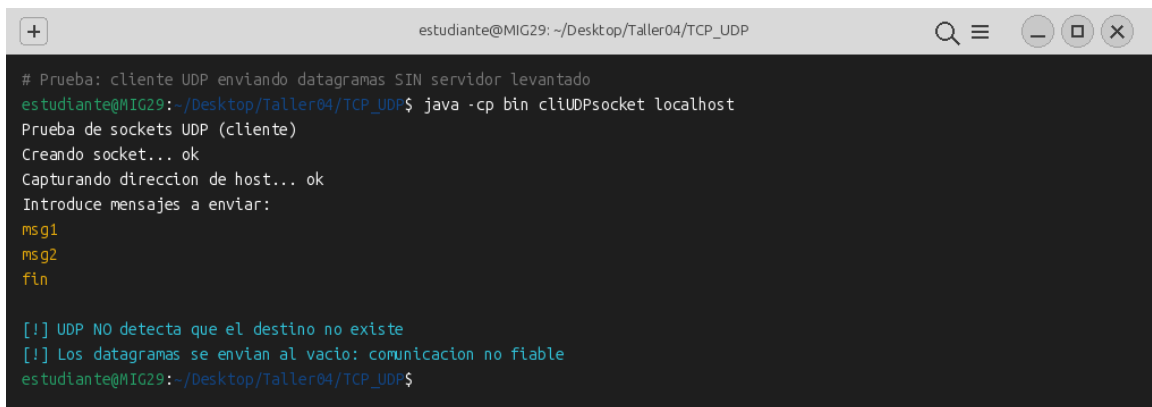


```

# Prueba: cliente TCP intentando conectarse SIN servidor levantado
estudiante@MIG29: ~/Desktop/Taller04/TCP_UDP$ java -cp bin cliTCPsocket localhost
Prueba de sockets TCP (CLIENTE)
Capturando direccion de host... ok
Creando socket... Connection refused

[!] TCP detecta inmediatamente que el destino NO existe
[!] La excepcion 'Connection refused' confirma orientacion a conexion
estudiante@MIG29: ~/Desktop/Taller04/TCP_UDP$

```

Figura 4: TCP detecta inmediatamente que el servidor no existe y lanza la excepción `Connection refused`.


```

# Prueba: cliente UDP enviando datagramas SIN servidor levantado
estudiante@MIG29: ~/Desktop/Taller04/TCP_UDP$ java -cp bin cliUDPsocket localhost
Prueba de sockets UDP (cliente)
Creando socket... ok
Capturando direccion de host... ok
Introduce mensajes a enviar:
msg1
msg2
fin

[!] UDP NO detecta que el destino no existe
[!] Los datagramas se envian al vacio: comunicacion no fiable
estudiante@MIG29: ~/Desktop/Taller04/TCP_UDP$

```

Figura 5: UDP no detecta la ausencia del servidor: los datagramas se envían al vacío sin reportar ningún error.

Esta prueba demuestra que TCP, al ser orientado a conexión, valida la existencia del extremo remoto antes de enviar cualquier dato, mientras que UDP confía en el modelo *best effort*: si los datagramas se pierden, la aplicación ni se entera.

4.7. Comparativa TCP vs UDP

Característica	TCP	UDP
Conexión previa	Sí (3-way handshake)	No
Confiabilidad	Sí (ACK + retransmisión)	No (best effort)
Orden de mensajes	Garantizado	No garantizado
Control de flujo	Sí	No
Control de errores	Sí	No
Velocidad	Más lenta	Más rápida
Sobrecarga	Alta	Baja
Pierde si destino cae	Detecta y lanza excepción	No detecta nada
Caso de uso típico	Web, correo, SSH	Streaming, DNS, juegos

Cuadro 2: Tabla comparativa de las principales características de TCP y UDP.

4.8. ¿Cuándo usar cada uno?

TCP es la opción adecuada cuando: la integridad y el orden de los datos son críticos. Ejemplos: navegación web (HTTP/HTTPS), correo electrónico (SMTP, IMAP), transferencia de archivos (FTP), conexiones remotas (SSH), bases de datos.

UDP es la opción adecuada cuando: la baja latencia importa más que la fiabilidad absoluta, o cuando un mensaje perdido ya no tiene valor. Ejemplos: streaming de video y audio en vivo, llamadas VoIP, juegos en línea, DNS, telemetría de sensores y descubrimiento de servicios en redes locales.

5. Parte II — Ejecución Secuencial vs Paralela con Hilos

5.1. Caso de Estudio

El ejercicio simula un supermercado con dos cajeras (*Cajera 1* y *Cajera 2*) que deben atender a dos clientes (*Cliente 1* y *Cliente 2*). Cada cliente trae un carro de compra representado por un arreglo de enteros, donde cada número indica los segundos que tarda en procesarse un producto en la caja:

- **Cliente 1:** { 2, 2, 1, 5, 2, 3 } → total = 15 segundos
- **Cliente 2:** { 1, 3, 5, 1, 1 } → total = 11 segundos

El procesamiento de cada producto se simula con `Thread.sleep(segundos * 1000)`, que bloquea el hilo durante el tiempo indicado.

5.2. Las Tres Versiones Implementadas

5.2.1. Versión Secuencial (*Main.java*)

El programa principal llama primero a `cajera1.procesarCompra(...)` y, cuando esa llamada termina, llama a `cajera2.procesarCompra(...)`. Como ambas se ejecutan en el mismo hilo (el hilo principal), la segunda cajera no puede empezar hasta que la primera haya terminado completamente.

5.2.2. Versión con *extends Thread* (*MainThread.java* y *CajeraThread.java*)

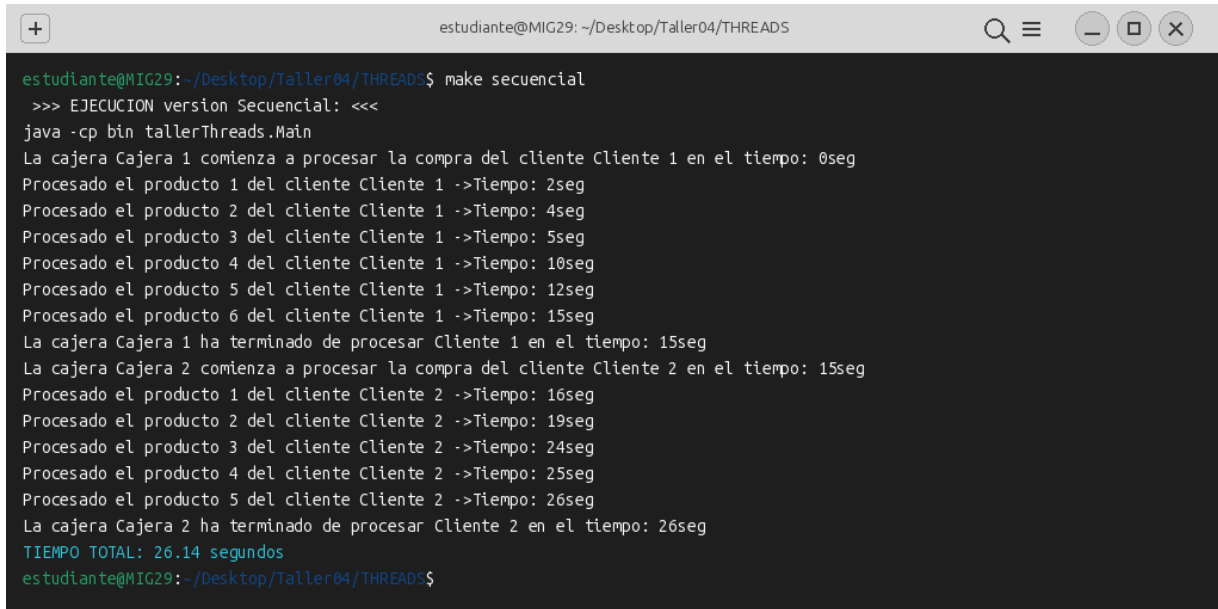
La clase `CajeraThread` hereda de `Thread` y sobrescribe el método `run()`. El programa principal crea dos instancias y las arranca con `start()`, lo que crea dos hilos independientes que se ejecutan en paralelo.

5.2.3. Versión con *implements Runnable* (*MainRunnable.java*)

La clase `MainRunnable` implementa la interfaz `Runnable`. Se crean dos objetos `Runnable` y se envuelven en sendos `Thread`. Esta es la variante recomendada porque desacopla el código a ejecutar del mecanismo de hilo, y porque una clase puede implementar varias interfaces pero sólo heredar de una.

5.3. Resultados Experimentales

5.3.1. Ejecución Secuencial



```
estudiante@MIG29:~/Desktop/Taller04/THREADS$ make secuencial
>>> EJECUCION version Secuencial: <<<
java -cp bin tallerThreads.Main
La cajera Cajera 1 comienza a procesar la compra del cliente Cliente 1 en el tiempo: 0seg
Procesado el producto 1 del cliente Cliente 1 ->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1 ->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1 ->Tiempo: 5seg
Procesado el producto 4 del cliente Cliente 1 ->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 1 ->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1 ->Tiempo: 15seg
La cajera Cajera 1 ha terminado de procesar Cliente 1 en el tiempo: 15seg
La cajera Cajera 2 comienza a procesar la compra del cliente Cliente 2 en el tiempo: 15seg
Procesado el producto 1 del cliente Cliente 2 ->Tiempo: 16seg
Procesado el producto 2 del cliente Cliente 2 ->Tiempo: 19seg
Procesado el producto 3 del cliente Cliente 2 ->Tiempo: 24seg
Procesado el producto 4 del cliente Cliente 2 ->Tiempo: 25seg
Procesado el producto 5 del cliente Cliente 2 ->Tiempo: 26seg
La cajera Cajera 2 ha terminado de procesar Cliente 2 en el tiempo: 26seg
TIEMPO TOTAL: 26.14 segundos
estudiante@MIG29:~/Desktop/Taller04/THREADS$
```

Figura 6: Salida de la versión secuencial: la cajera 2 arranca en el segundo 15, justo cuando la cajera 1 termina. Tiempo total: 26 s.

5.3.2. Ejecución con Hilos (*extends Thread*)

```
estudiante@MIG29: ~/Desktop/Taller04/THREADS$ make threads
>>> EJECUCION version con Threads (extends Thread): <<<
java -cp bin tallerThreads.MainThread
La cajera Cajera 1 comienza a procesar la compra del cliente Cliente 1 en el tiempo: 0seg
La cajera Cajera 2 comienza a procesar la compra del cliente Cliente 2 en el tiempo: 0seg
Procesado el producto 1 del cliente Cliente 2 ->Tiempo: 1seg
Procesado el producto 1 del cliente Cliente 1 ->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1 ->Tiempo: 4seg
Procesado el producto 2 del cliente Cliente 2 ->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1 ->Tiempo: 5seg
Procesado el producto 3 del cliente Cliente 2 ->Tiempo: 9seg
Procesado el producto 4 del cliente Cliente 1 ->Tiempo: 10seg
Procesado el producto 4 del cliente Cliente 2 ->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 2 ->Tiempo: 11seg
La cajera Cajera 2 ha terminado de procesar Cliente 2 en el tiempo: 11seg
Procesado el producto 5 del cliente Cliente 1 ->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1 ->Tiempo: 15seg
La cajera Cajera 1 ha terminado de procesar Cliente 1 en el tiempo: 15seg
TIEMPO TOTAL: 15.12 segundos
estudiante@MIG29: ~/Desktop/Taller04/THREADS$
```

Figura 7: Las dos cajas arrancan en el segundo 0 y trabajan en paralelo. La caja 2 termina a los 11 s y la caja 1 a los 15 s. Tiempo total: 15 s.

5.3.3. Ejecución con Runnable

```

estudiante@MIG29: ~/Desktop/Taller04/THREADS
+
estudiante@MIG29:~/Desktop/Taller04/THREADS$ make runnable
>>> EJECUCION version con Runnable (implements Runnable): <<<
java -cp bin tallerThreads.MainRunnable
La cajera Cajera 1 comienza a procesar la compra del cliente Cliente 1 en el tiempo: 0seg
La cajera Cajera 2 comienza a procesar la compra del cliente Cliente 2 en el tiempo: 0seg
Procesado el producto 1 del cliente Cliente 2 ->Tiempo: 1seg
Procesado el producto 1 del cliente Cliente 1 ->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1 ->Tiempo: 4seg
Procesado el producto 2 del cliente Cliente 2 ->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1 ->Tiempo: 5seg
Procesado el producto 3 del cliente Cliente 2 ->Tiempo: 9seg
Procesado el producto 4 del cliente Cliente 1 ->Tiempo: 10seg
Procesado el producto 4 del cliente Cliente 2 ->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 2 ->Tiempo: 11seg
La cajera Cajera 2 ha terminado de procesar Cliente 2 en el tiempo: 11seg
Procesado el producto 5 del cliente Cliente 1 ->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1 ->Tiempo: 15seg
La cajera Cajera 1 ha terminado de procesar Cliente 1 en el tiempo: 15seg
TIEMPO TOTAL: 15.12 segundos
estudiante@MIG29:~/Desktop/Taller04/THREADS$

```

Figura 8: La versión con Runnable produce el mismo comportamiento y el mismo tiempo total que la versión con extends Thread.

5.4. Tabla Resumen de Tiempos

Versión	Cliente 1	Cliente 2	Tiempo total
Secuencial (Main)	15 s	26 s	26.14 s
Threads (MainThread)	15 s	11 s	15.12 s
Runnable (MainRunnable)	15 s	11 s	15.12 s

Cuadro 3: Tiempos en que cada cliente termina de ser atendido y tiempo total de ejecución del programa para cada versión.

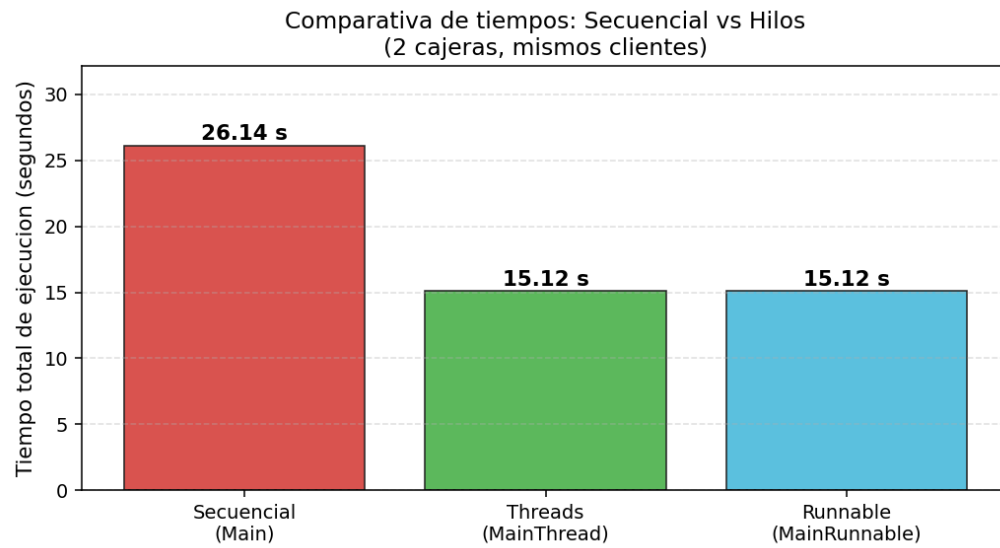


Figura 9: Comparativa visual de los tiempos totales de ejecución de las tres versiones.

5.5. Análisis de los Resultados

La versión paralela (sea con *extends Thread* o con *Runnable*) redujo el tiempo total de **26.14 s a 15.12 s**, lo que representa una mejora cercana al **42 %**. La razón es clara: en la versión secuencial el tiempo total es la *suma* de los tiempos de los dos clientes ($15 + 11 = 26$ s), mientras que en la versión paralela el tiempo total es el *máximo* entre los dos ($\text{máx}(15, 11) = 15$ s).

Las versiones con **Thread** y con **Runnable** dieron exactamente el mismo tiempo, lo cual es coherente: ambas usan internamente un hilo del sistema operativo, y la única diferencia es cómo se modela la clase. La ventaja de **Runnable** es de diseño: como Java no permite herencia múltiple, si una clase ya hereda de otra, no podría extender **Thread**; implementar **Runnable** resuelve ese problema.

También se observa que en la versión paralela el orden de los mensajes en la salida no es estrictamente determinista: a veces aparece primero el producto del Cliente 2 y luego el del Cliente 1, dependiendo de cómo el planificador del SO asigne CPU a los hilos. Esta es una característica fundamental de la programación concurrente: el orden de ejecución es no determinista.

6. Conclusiones

1. **TCP y UDP no son intercambiables.** Cada uno tiene un caso de uso claro: TCP cuando importa la integridad y orden de los datos, UDP cuando importa la velocidad y la pérdida de algún mensaje es tolerable.
2. **La diferencia más observable** entre TCP y UDP en una prueba real es cómo reaccionan ante la ausencia del servidor: TCP lanza inmediatamente **Connection refused**, mientras que UDP envía datagramas al vacío sin reportar nada.
3. **El paralelismo con hilos puede aportar mejoras significativas** cuando las tareas son independientes y dominadas por tiempos de espera (E/S, **sleep**, espera de red). En este taller se logró una mejora del 42 %.
4. **Implements Runnable es preferible a extends Thread** en la mayoría de casos reales por flexibilidad de diseño, aunque ambos producen los mismos resultados de rendimiento.
5. **La programación concurrente introduce no determinismo:** el orden exacto de las salidas no se puede predecir, lo que tiene implicaciones para el diseño y las pruebas de sistemas distribuidos.
6. **Estos dos temas son la base de los sistemas distribuidos:** los sockets permiten que los procesos hablen entre sí a través de la red, y los hilos permiten que un mismo proceso atienda múltiples conexiones o tareas en paralelo. Combinándolos se construyen servidores escalables.

7. Observaciones

- Los archivos originales del taller traían errores de compilación (cabeceras con caracteres ilegales y faltaba el `import java.net.*; import java.io.*;` en los TCP); se corrigieron antes de ejecutar.
- Las pruebas se realizaron con cliente y servidor en la misma máquina (`localhost`); en una red real, las diferencias entre TCP y UDP serían aún más pronunciadas debido a la latencia y a la posibilidad real de pérdida de paquetes.
- El bucle del servidor TCP original era infinito (`while(1>0)`); se ajustó para que termine cuando el cliente envía `fin`, igual que el UDP.

Referencias

- [1] Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2012). *Distributed Systems: Concepts and Design* (5th ed.). Addison-Wesley.
- [2] Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed Systems* (3rd ed.). Pearson.
- [3] Oracle Corporation. (2024). *Java Platform Standard Edition 21 Documentation: Package java.net*. Recuperado de <https://docs.oracle.com/en/java/javase/21/>
- [4] Postel, J. (1980). RFC 768 — *User Datagram Protocol*.
- [5] Postel, J. (1981). RFC 793 — *Transmission Control Protocol*.
- [6] Goetz, B., et al. (2006). *Java Concurrency in Practice*. Addison-Wesley.
- [7] Corredor, J. (2026). *Taller 04: Hilos y Sockets*. Pontificia Universidad Javeriana, Departamento de Ingeniería de Sistemas.